Singly Linked List –
Traverse List Inorder2
and Time Complexity.

Traversing List InOrder means traversing list starting from first index to last index.

Program:

```
void TraverseListInorder2(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {
  for (Node **current = head; *current != nullptr; current = &((*current)->next))
        {
            cout << (*current)->data << " ";
        }
        cout << endl;
    }
}

….
case 23:
        {
            cout << endl;
            cout << endl;
            cout << "Traverse List In Order 2:(For Loop)" << endl;
            TraverseListInorder2(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

Program Analysis:

Function Call:

$Function\ call\ occurs\ when\ Case\ 23\ is\ initiated\ from\ Switch - Case$:

$cout \ldots\ldots TraversingListInorder2(\&head);$

$There\ are\ two\ criteria\ of\ input\ Linked\ List$:

1)*Linked List is Empty*:

$Pictorially\ depicted\ as$:

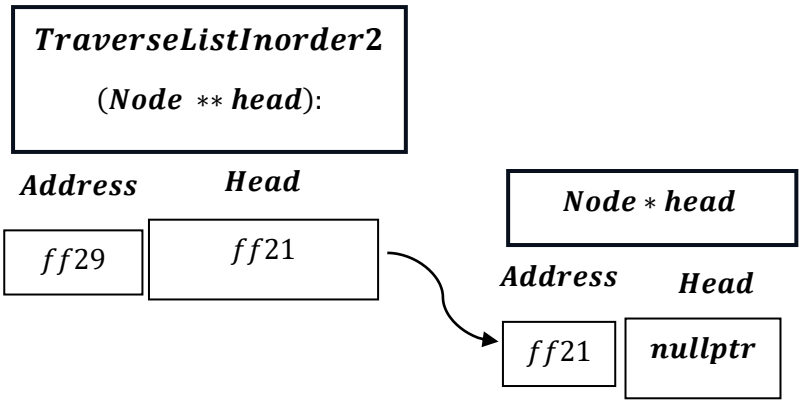
$Node * head$

$Address$	$Head$
$ff21$	$nullptr$

```
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
```

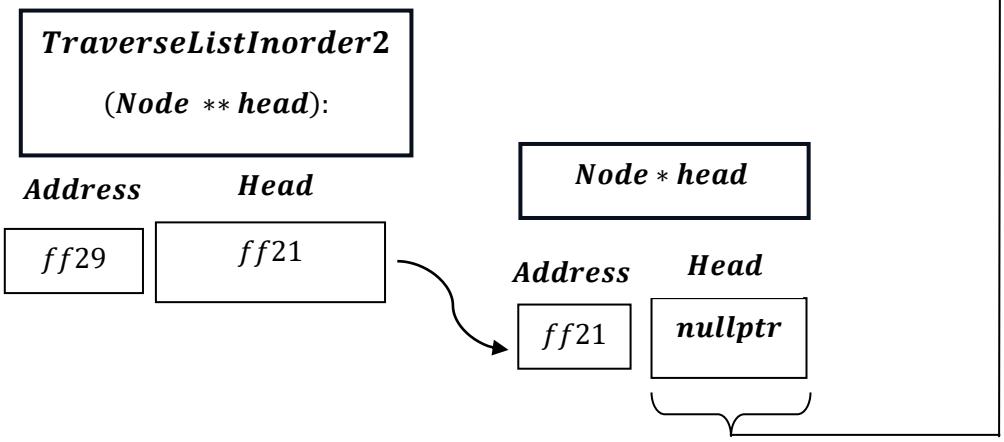
$TraverseListInorder2(\&head: ff21);$

```
void TraverseListInorder2(Node **head) {….}
```



$if(*head == nullptr)$ *if it is true*

If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the `TraverseListInorder2` function.

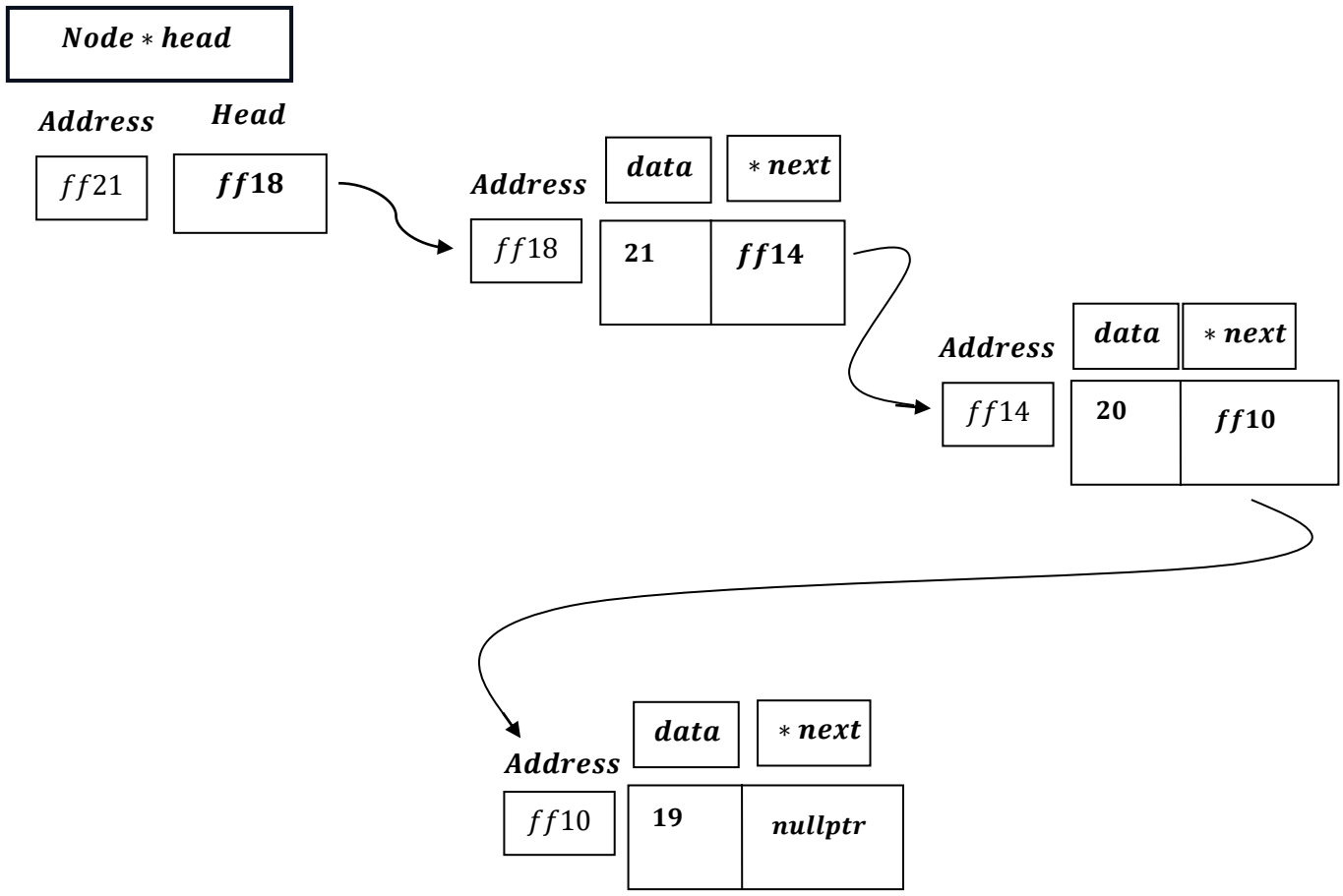


If List is Empty , then , it prints a message : `Error: List is empty.` and returns nothing as function has void data type . And function ends.

---

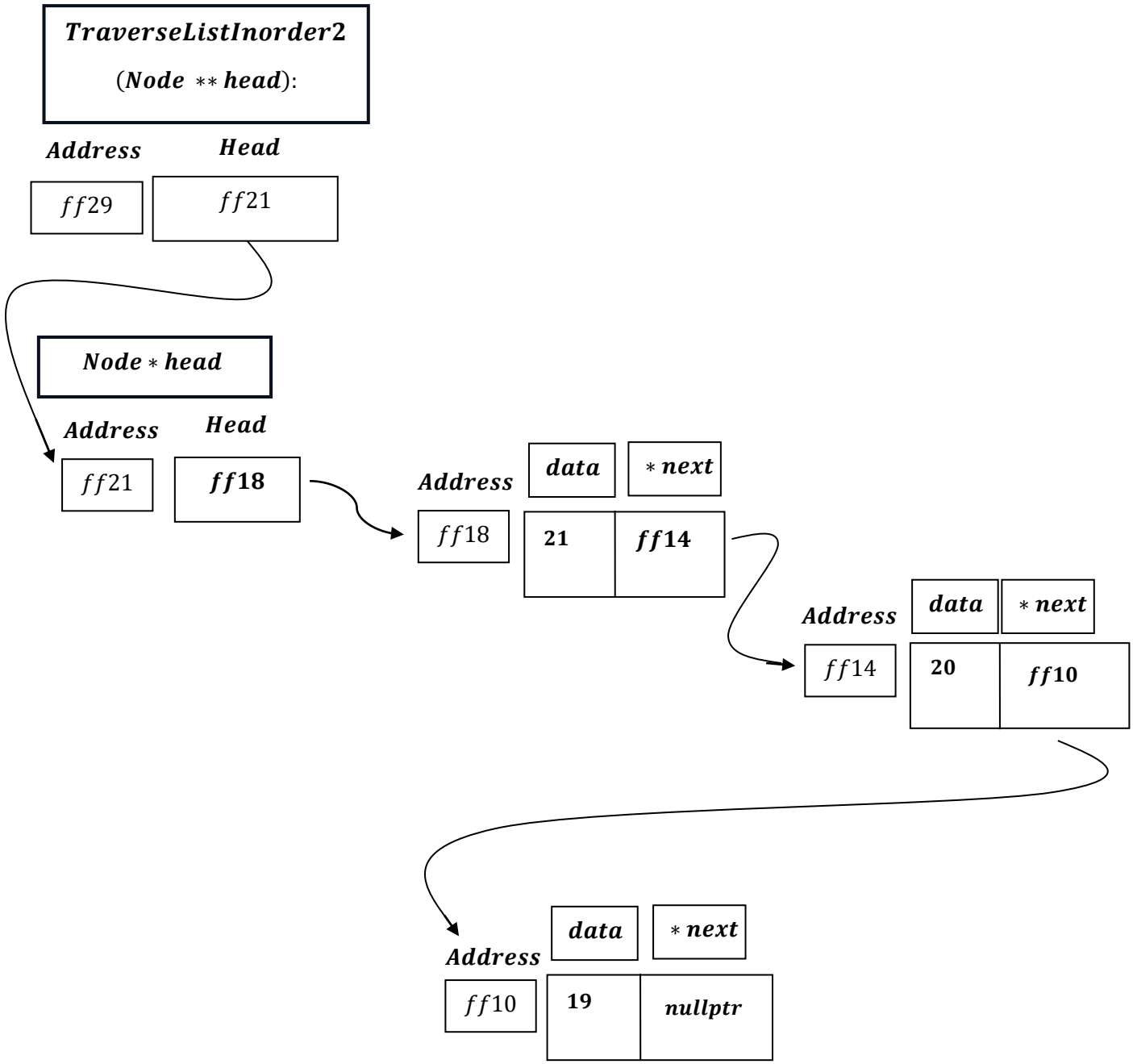
2)*Linked List is not Empty*:

Pictorially depicted as:



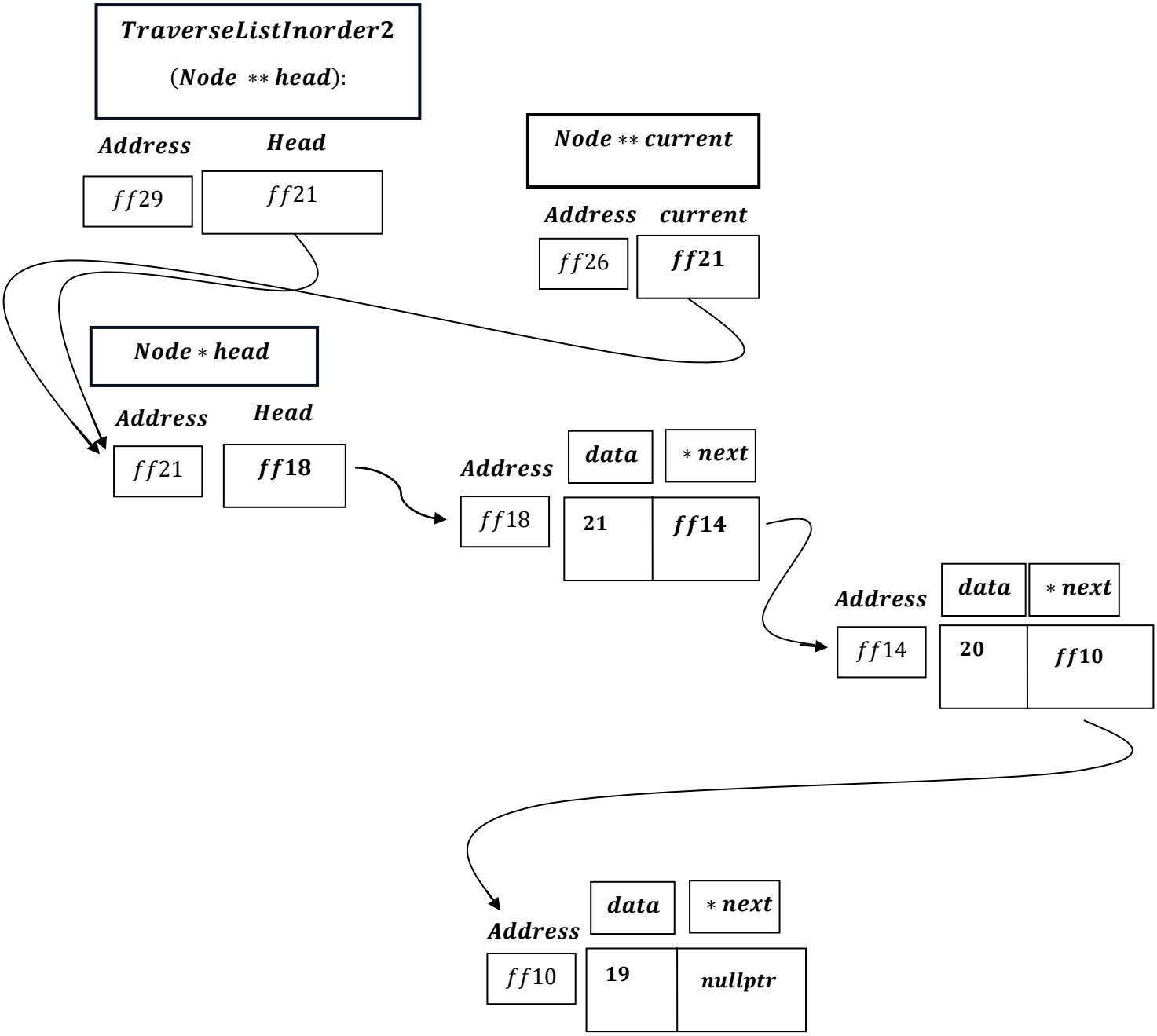

```
void TraverseListInorder2(Node **head)
{ ……
    else
    {
  for (Node **current = head; *current != nullptr; current = &((*current)->next))
        {
            cout << (*current)->data << " ";
        }
        cout << endl;
    }
}
```

```
void TraverseListInorder(Node **head){….}
```



Now , we have for loop initialization running:

```
for (Node **current = head;….){……}
```



$Node * \underline{*current}$ pointer, Here $\underline{Node *}$ points to address: $ff18$, where $\underline{current}$ points to address : $ff21$.

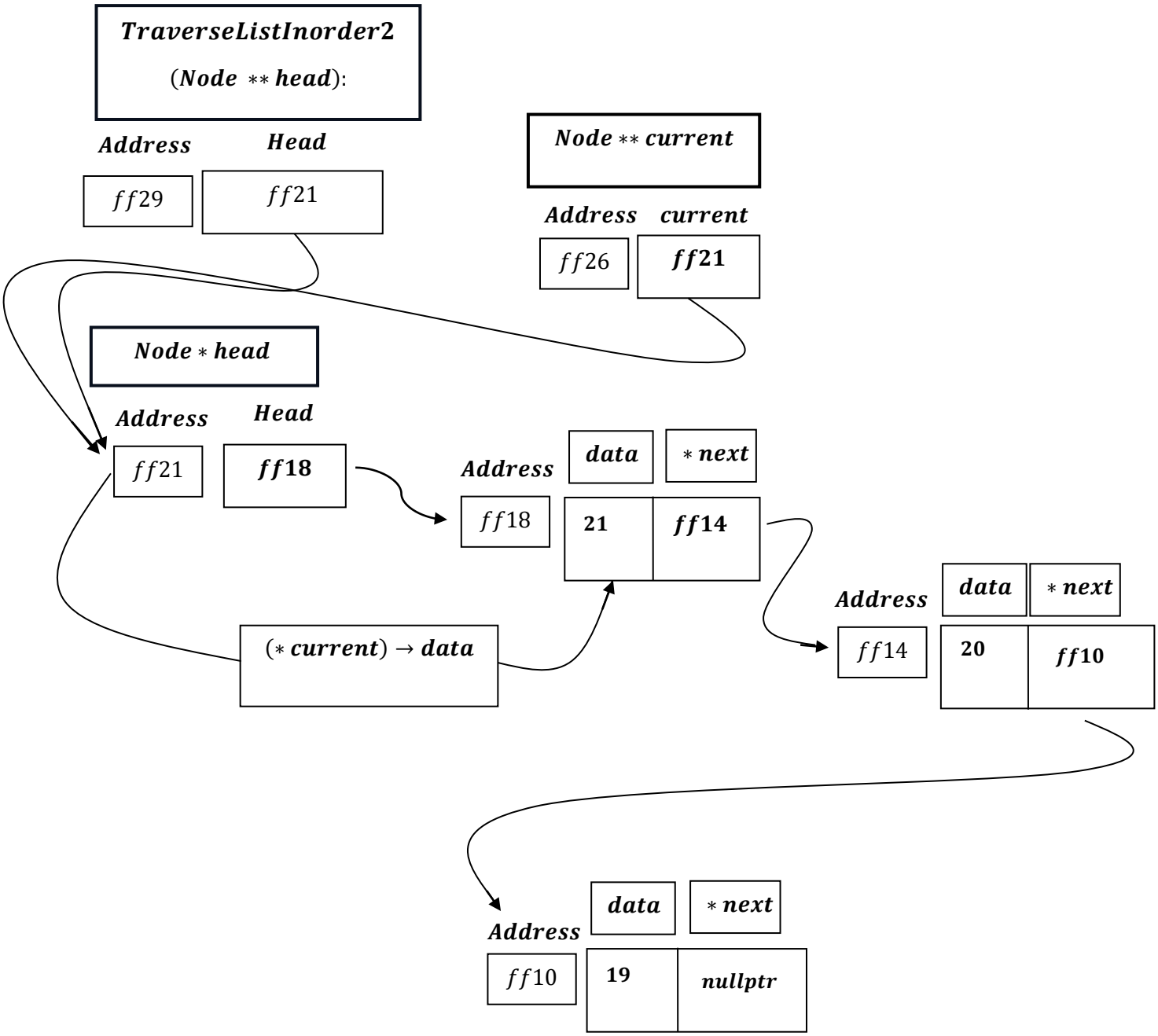
Now, we have for loop condition running :

```
        for (….; *current != nullptr; ….)
        {
            cout << (*current)->data << " ";
        }
```

$for(\ldots; *current != nullptr; \ldots)\{$

$cout \ll (*current) \rightarrow data \ll$ ``  ``; $Output: 21$

$\}$



$(*current)$, here braces prioritize the pointer current variable, i.e.

$Node*$ $\underline{*current}$ pointer, and dereference it, to get the

$Node*$ it points to. Here $\underline{Node*}$ points to address: $ff18$, where

$\underline{current}$ points to address : $ff21$. Hence as $\underline{*current}$ pointer gets deferenced by $*$ (dereferencing operator), i.e. now

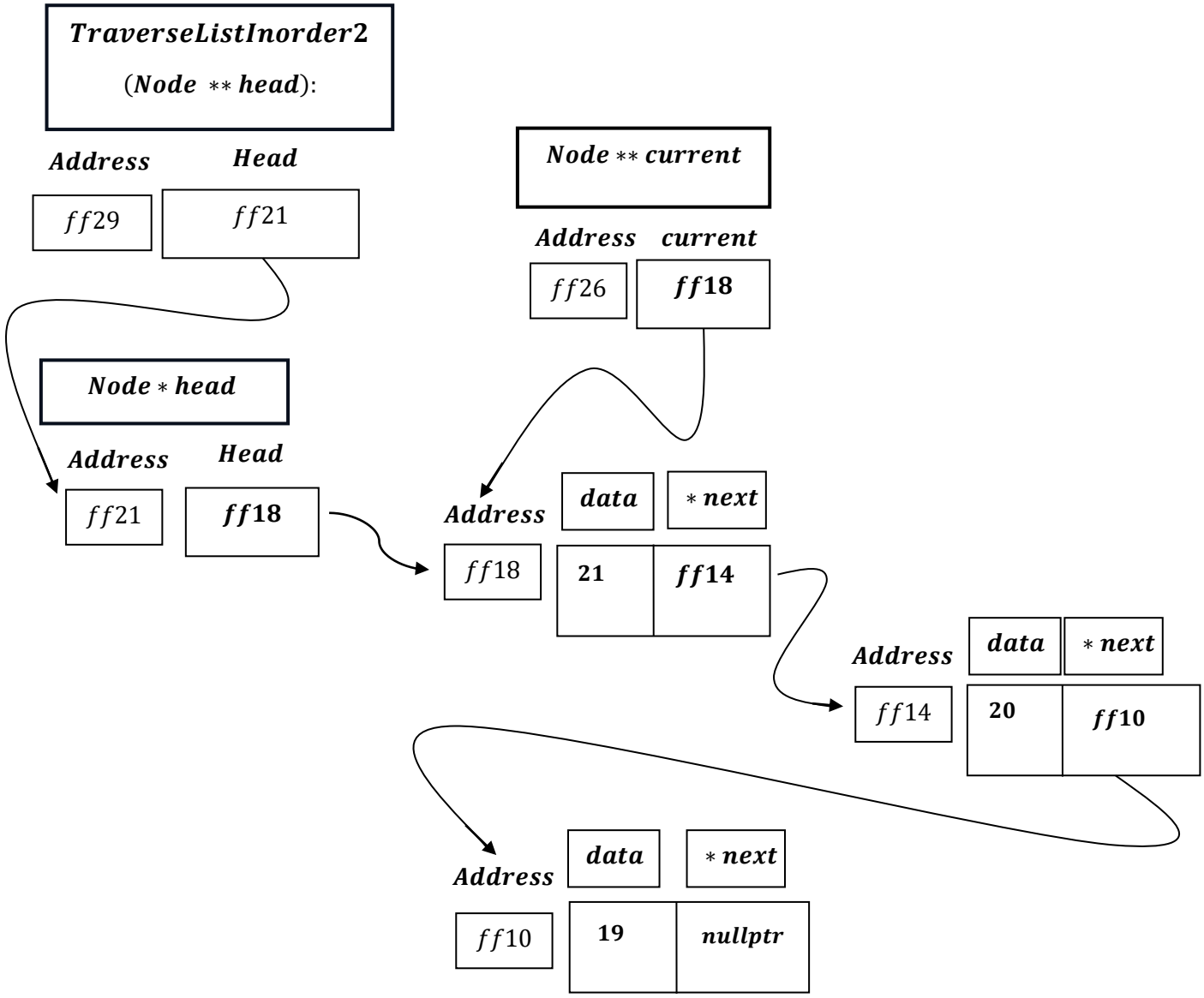
no longer pointing to address : $ff21$, hence, now

$Node*$ pointed address is referenced i.e. $ff18$ and it gives us `data`

stored in address $ff18$ i.e., 21.

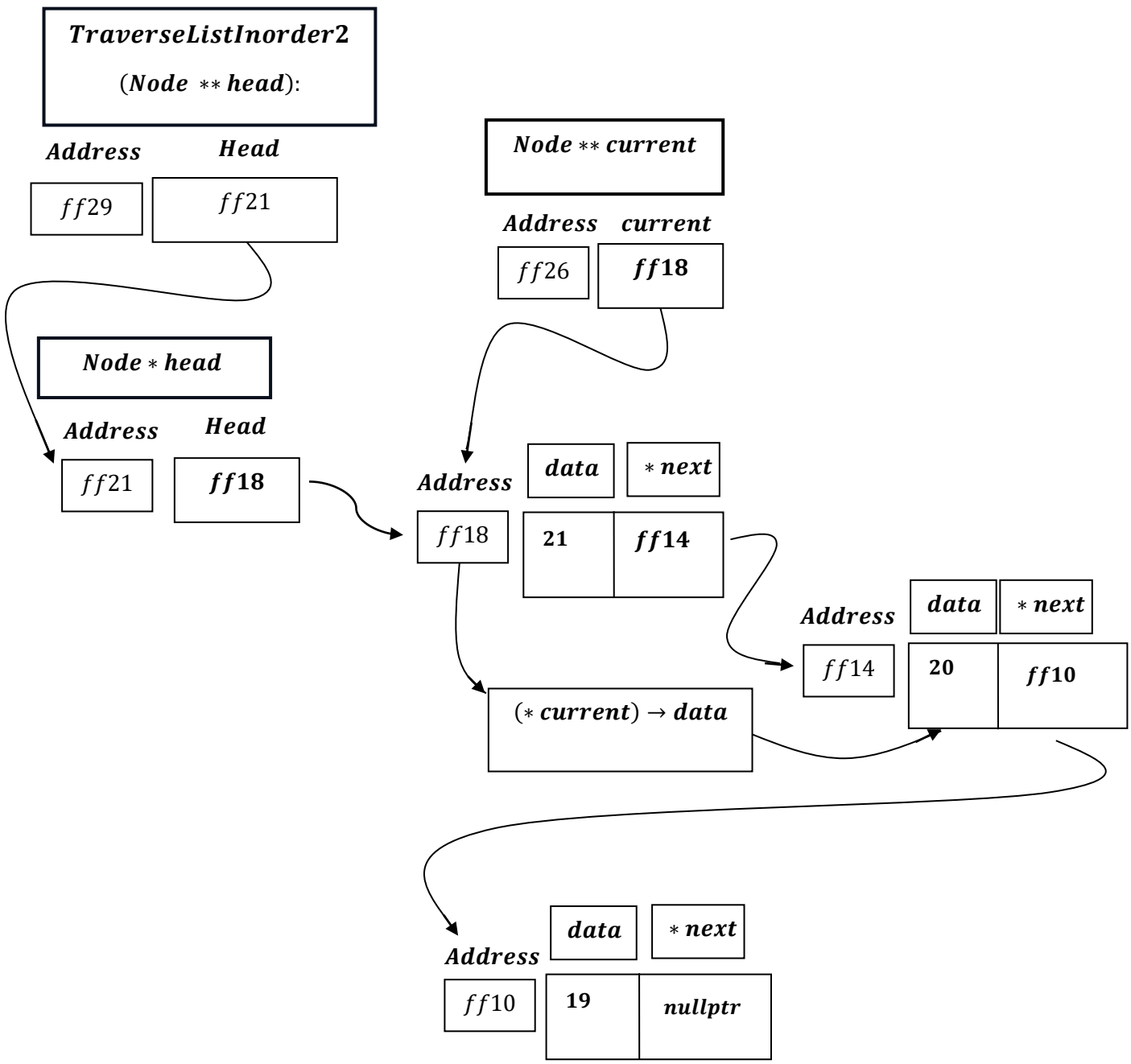
Now, updation:

```
for (…; ….; current = &((*current)->next)){….}
```



$During\ the\ first\ iteration, (*current) dereferences\ current\ to\ get\ the\ Node\ *$
$it\ points\ to, which\ is\ the\ first\ node\ in\ the\ list. The\ expression\ (*current) \rightarrow next$
$, where\ current\ points\ to\ Node * Head : ff21, but\ (*current)\ dereferences\ first\ and$
$now\ Node * points\ to\ first\ node, hence\ (*current) \rightarrow next, will\ point\ to\ next\ pointer$
$of\ first\ node\ i.e. ff14\ . The\ \&\ operator\ then\ takes\ the\ address\ where\ this$
$next\ pointer\ is\ stored\ i.e. the\ address: ff18, where\ ff14\ is\ stored\ . Hence,$
$current = \&\big((*current) \rightarrow next\big) = ff18\ under\ which\ ff14\ is\ stored.$

$for(\ldots; *current\ != nullptr; \ldots)\{$
$\quad cout \ll (*current) \rightarrow data \ll \text{``}\ \text{''}; Output : 20$
$\}$



$(*\ current)$, here braces prioritize the pointer current variable, i.e.

$Node\ *\ \underline{*\ current}$ pointer, and dereference it, to get the

$Node\ *$ it points to. Here $\underline{Node\ *}$ points to address: $ff14$, where

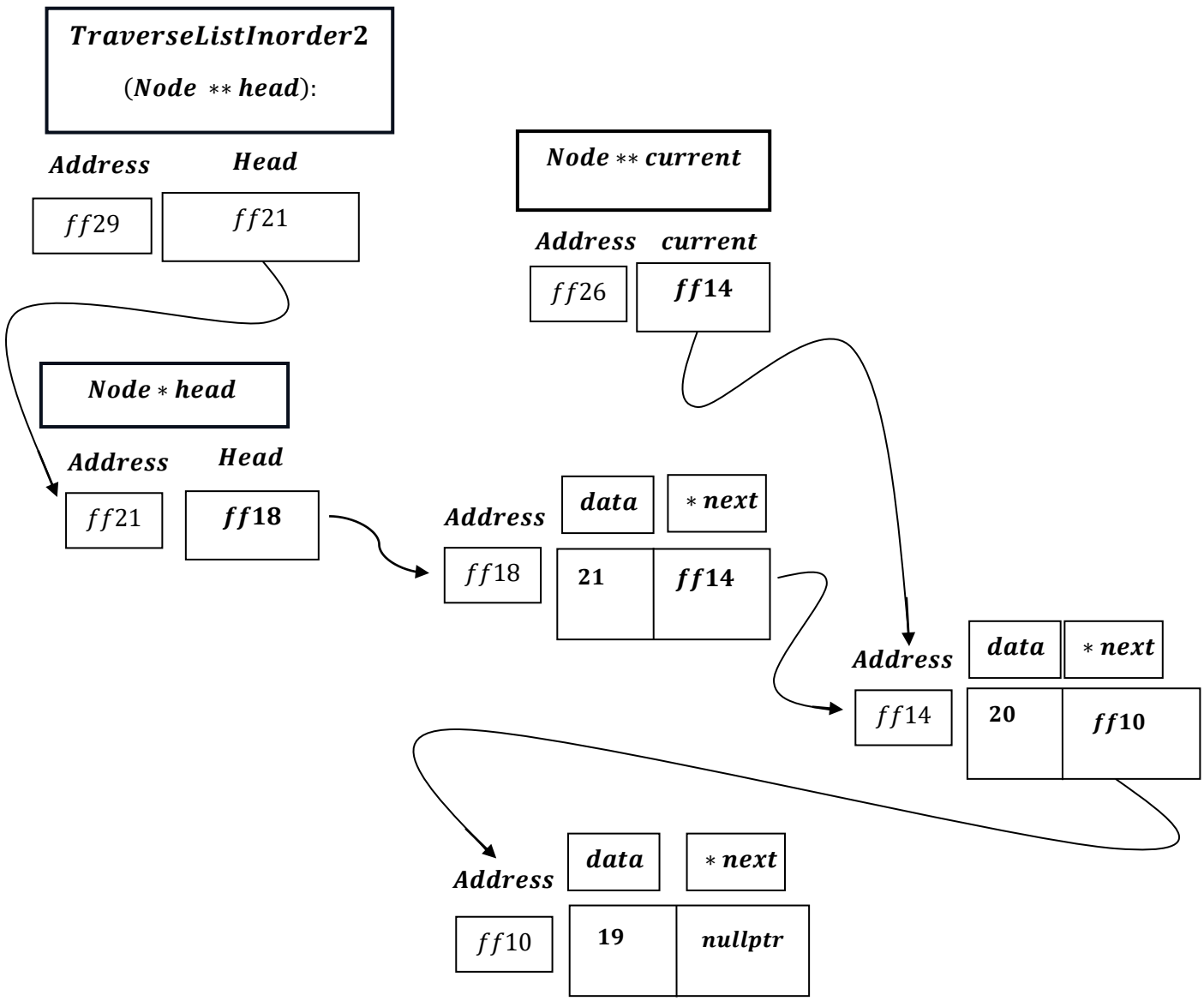
$\underline{current}$ points to address : $ff18$. Hence as $\underline{*\ current}$ pointergets deferenced by $*$ (dereferencing operator), i.e. now

no longer pointing to address : $ff18$, hence, now

$Node\ *$ pointed address is referenced i.e. $ff14$ and it gives us `data`

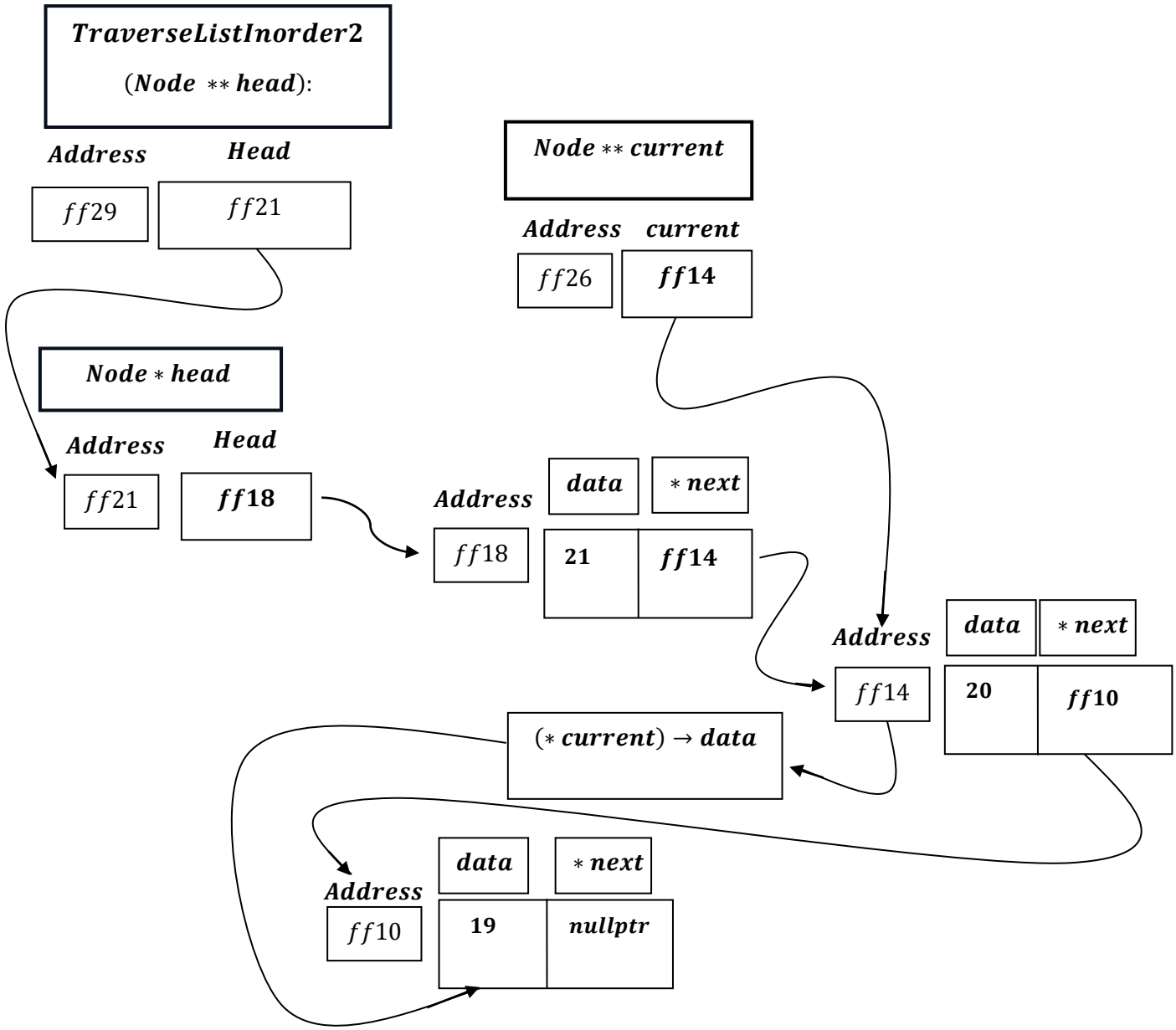
stored in address $ff14$ i.e., 20.

```
for (…; ….; current = &((*current)->next)){….}
```



$During\ the\ second\ iteration, (*current) dereferences\ current\ to\ get\ the\ Node\ *$
$it\ points\ to, which\ is\ the\ second\ node\ in\ the\ list. The\ expression\ (*current) \rightarrow next$
$, where\ current\ points\ to\ address: ff18, but\ (*current)\ dereferences\ first\ and$
$now\ Node * points\ to\ second\ node, hence\ (*current) \rightarrow next, will\ point\ to$
$next\ pointer\ of\ second\ node\ i.e.\ ff10\ .The\ \&\ operator\ then\ takes\ the\ address\ where$
$this\ next\ pointer\ is\ stored\ i.e.\ the\ address: ff14, where\ ff10\ is\ stored\ .Hence,$
$current = \&\big((*current) \rightarrow next\big) = ff14\ under\ which\ ff10\ is\ stored.$

```
for(...;*current != nullptr; ...){
        cout << (*current) -> data << ``  ``; Output : 20
}
```



$(*\ current)$, here braces prioritize the pointer current variable, i.e.

$Node*$ $\underline{*current}$ pointer, and dereference it, to get the

$Node*$ it points to. Here $\underline{Node*}$ points to address: $ff10$, where

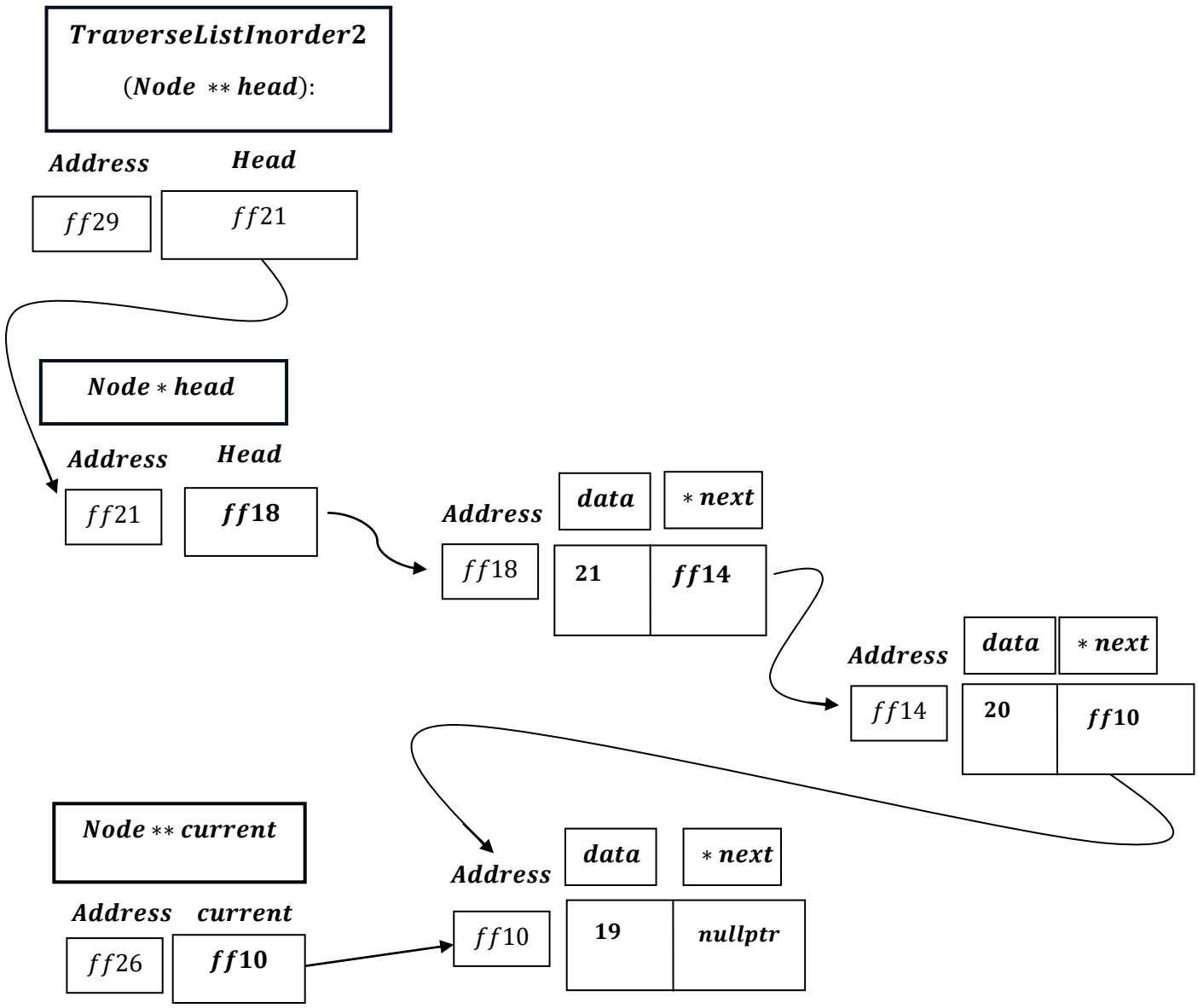
$\underline{current}$ points to address : $ff14$. Hence as $\underline{*current}$
pointer gets deferenced by $*$ (dereferencing operator), i.e. now

no longer pointing to address : $ff14$, hence, now

$Node*$ pointed address is referenced i.e. $ff10$ and it gives us `data`

stored in address $ff10$ i.e., 19.

```
for (…; …..; current = &((*current)->next)){….}
```



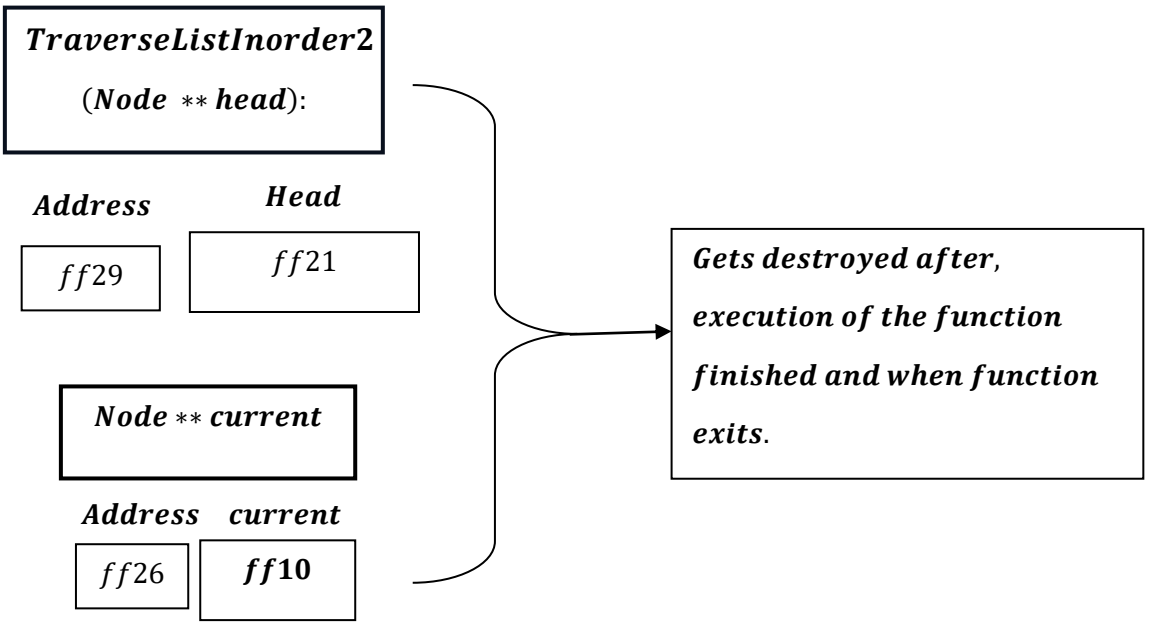
$During\ the\ third\ iteration, (*current) dereferences\ current\ to\ get\ the\ Node\ *$
$it\ points\ to, which\ is\ the\ third\ node\ in\ the\ list. The\ expression\ (*current) \rightarrow next$
$, where\ current\ points\ to\ address: ff14, but\ (*current)\ dereferences\ first\ and$
$now\ Node * points\ to\ third\ node, hence\ (*current) \rightarrow next, will\ point\ to$
$next\ pointer\ of\ third\ node\ i.e. ``nullptr``\ . The\ \&\ operator\ then\ takes\ the\ address$
$where\ this\ nullptr\ is\ stored\ i.e. the\ address: ff10, where\ ``nullptr``\ is\ stored\ .$
$Hence, current = \&\big((*current) \rightarrow next\big) = ff10\ under\ which\ ``nullptr``\ is\ stored.$

```
for(...;*current != nullptr; ...){
        cout << (*current) -> data << " ";Output : 20
  }
```

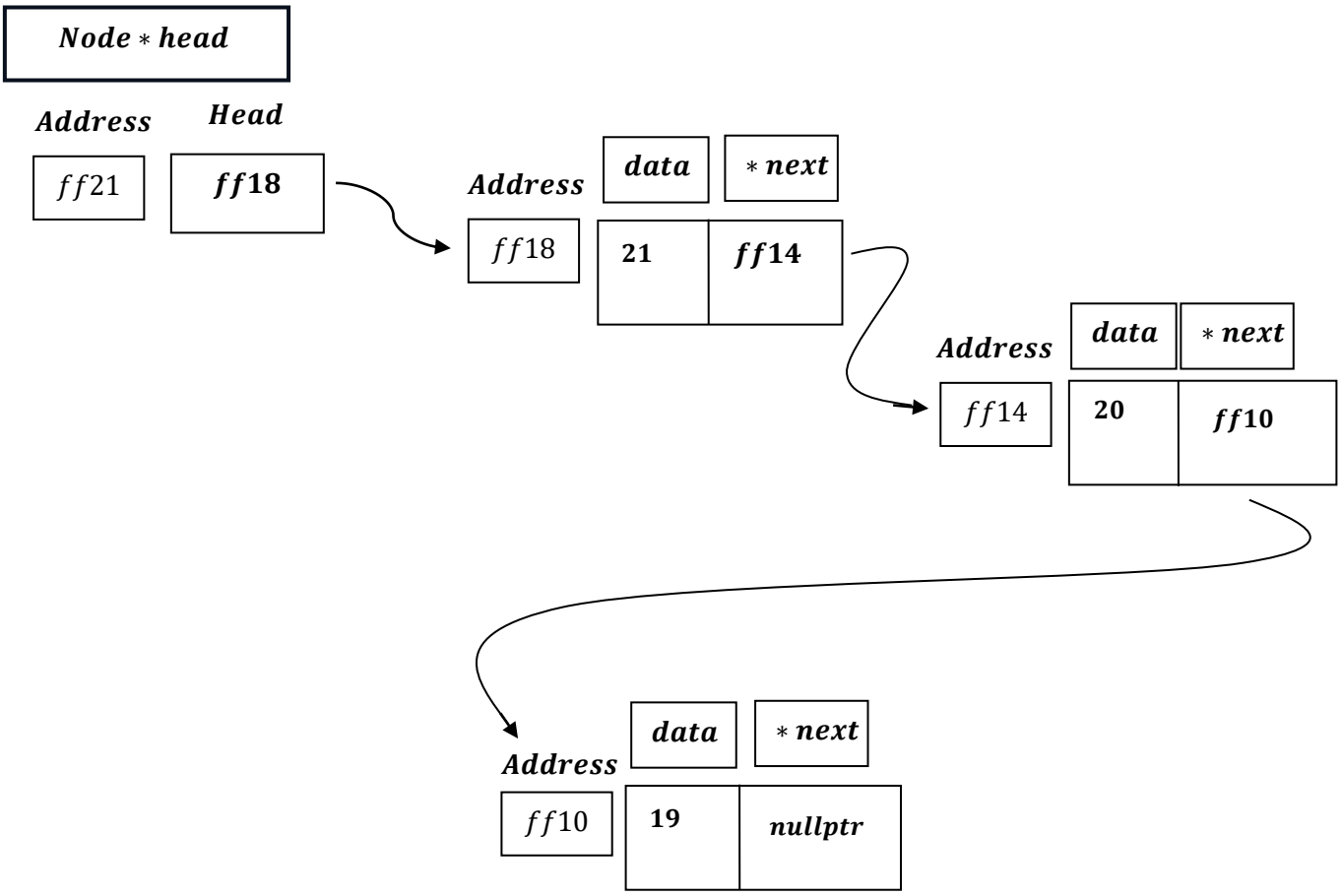
$*current, due\ to\ conditional\ checking, hence\ no\ braces\ are\ needed\ to$
$prioritize\ the\ pointer\ current\ variable, i.e. Node * \underline{*current}\ pointer, and$
$dereference\ it\ , to\ get\ the\ Node\ *\ it\ points\ to. Here\ \underline{Node *}\ points\ to$
$``nullptr``, where\ \underline{current}\ points\ to\ address: ff10. Hence\ as\ \underline{*current}$
$pointergets\ deferenced\ by\ *\ (dereferencing\ operator), i.e.\ now\ no\ longer$
$pointing\ to\ address: ff10, hence\ , now\ Node * pointed\ address\ is$
$referenced\ i.e. ``nullptr``.$

$Now, as\ soon\ as\ we\ get\ ``nullptr``, the\ condition\ of\ for\ loop\ fails\ and\ exits$
$from\ the\ loop.$

Now, as the function get finished, all temporary variable created earlier in process for the program will get destroyed.



And we are left with:



$Now, `TraverseListInorder2` \ step\ by\ step\ explanation:$

$1. Check\ if\ the\ List\ is\ Empty:$

- $The\ function\ first\ checks\ if\ the\ list\ is\ empty\ by\ dereferencing\ head\ to\ see\ if\ it\ points\ to\ nullptr. If\ *head\ is\ nullptr, it\ means\ the\ list\ is\ empty, and\ the\ function\ prints\ "List\ is\ empty."\ and\ returns\ immediately.$

$2. Traverse\ the\ List:$

- $Initialization: Node\ **current = head;\ initializes\ current\ to\ point\ to\ the\ head\ of\ the\ list.$
- $Condition: *current != nullptr;\ continues\ the\ loop\ as\ long\ as\ current\ points\ to\ a\ non-null\ node.$
- $Updation: current = &((*current) \rightarrow next)\ updates\ current\ to\ point\ to\ the\ address\ of\ the\ next\ pointer\ of\ the\ current\ node. This\ effectively\ moves\ current\ to\ the\ next\ node\ in\ the\ list.$

$3. Print\ Node\ Data:$

- $Inside\ the\ loop, cout \ll (*current) \rightarrow data \ll ;\ prints\ the\ data\ of\ the\ current\ node\ followed\ by\ a\ space.$

$4. End\ of\ List:$

- $After\ the\ loop\ completes, cout \ll endl;\ prints\ a\ newline\ character\ to\ end\ the\ output.$

Time Complexity of

Traverse List Inorder2

```
void TraverseListInorder2(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {
  for (Node **current = head; *current != nullptr; current = &((*current)->next))
        {
            cout << (*current)->data << " ";
        }
        cout << endl;
    }
}
```

$O(1)$

runs `n` times

runs `n` times i.e. $O(n)$.

$O(1)$

Best Case (When List is Empty)

When List is Empty, then, as soon as it enters the function: TraverseListInorder2, if checks whether head pointer pointed to ``nullptr`` or not , if so then outputs: ``List Is Empty`` and returns nothing , and then exits from function. This takes, constant `c` amount of time i.e. $\Omega(1)$.

Hence when List is Empty, we get our best case : $\Omega(1)$.

$\underline{Worst\ Case\ (When\ List\ Is\ Not\ Empty)}$

$When\ list\ is\ not\ empty\ , it\ traverse\ whole\ list\ , assuming\ `n`\ is\ the\ length\ of\ the\ List, then\ the\ loop\ traverse\ `n`\ no. of\ nodes\ present\ in\ the\ list.$

$Where\ statement : cout \ll (* current) \rightarrow data \ll ``\ ``; outputs\ , `n`\ times.$

$Hence\ total\ number\ of\ statements\ in\ the\ loop\ runs\ `n`\ times = O(n).$

$\underline{Average\ Case(Average\ Case = Worst\ Case)}$

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list\ of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data, the\ time\ complexity\ is\ (O(n)), where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list\ of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data, the\ time\ complexity\ is\ (O(n)), where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list\ of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data,\ the\ time\ complexity\ is\ (O(n)), where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq n \leq c_2 \times n$

$Singly\ Linked\ List$	$Cases$	$Time\ Complexity$
$Traversal\ List\ Inorder2$	$1. Best\ Case$	$\Omega(1)$
	$2. Worst\ Case$	$O(n)$
	$3. Average\ Case = Worst\ Case$	$\Theta(n)$

****